

Explicit Euler

First-order forward discretization

1 Numerical map

For an initial-value problem

$$\dot{z} = f(t, z), \quad z(t_n) = z_n,$$

Taylor expansion gives

$$z(t_n + h) = z_n + hf(t_n, z_n) + \mathcal{O}(h^2).$$

Explicit Euler discards the remainder and defines

$$\Phi_{h,t_n}(z_n) = z_n + hf(t_n, z_n).$$

The method requires one vector-field evaluation, has local truncation error $\mathcal{O}(h^2)$ and, under the standard stability and smoothness assumptions, global error $\mathcal{O}(h)$.

2 Stability

For the test equation $y' = \lambda y$ the amplification factor is

$$R(\xi) = 1 + \xi, \quad \xi = h\lambda.$$

Absolute stability requires $|1 + \xi| \leq 1$, the disk centered at -1 with radius one. The bounded stability region makes this method unsuitable for stiff problems unless the step is very small.

3 Hamiltonian interpretation

For a canonical Hamiltonian system with Jacobian $A = Df(z_n)$, the step tangent is $D\Phi = I + hA$. Even when A is Hamiltonian, $(D\Phi)^T \Omega D\Phi = \Omega + h^2 A^T \Omega A$ in general. Explicit Euler is therefore neither symmetric nor symplectic and does not preserve energy in general. It serves as a transparent first-order baseline for accuracy and geometric diagnostics.

4 Implemented configuration

One forward Euler stage. Energy tracking is optional and passive. Forward Euler evaluates the passive derivative at the start of the accepted step. The common state conventions are specified below.

5 Implementation correspondence

The method is implemented in `src/methods/classical/euler.py`. The fixed-grid execution lifecycle is documented in the sibling `simulation/` directory; shared physical dynamics are documented outside the model tree.

6 Four state formulations and passive energy tracking

For one planar particle the supported internal representations are

physical	$z = (x, y)$	$\mathbb{R}^2$,
physical with energy	(z, t, κ)	$\mathbb{R}^4$,
duplicated	$(u, v), \quad u, v \in \mathbb{R}^2$	$\mathbb{R}^4$,
duplicated with energy	(u, v, t, κ)	$\mathbb{R}^6$.

Classical methods use the physical rows; ABBA and BM4 use the duplicated rows. `track_energy` selects the energy variant. For N planar particles the respective dimensions are $2N$, $4N$, $4N$, and $6N$: one time entry and one momentum per particle. All time entries equal the integration time. Classical full-cyclotron runs instead use $4N$ or $6N$ entries. Accepted spatial copies coincide; only one physical copy is exported. Every coordinate block contains N values in component-major order. The formulation validates particle times and aligns output round-off; the integrator does not interpret coordinate indices.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, z), \quad \kappa(t_0) = 0, \quad \Delta K(t) = H(t, z(t)) + \kappa(t) - H(t_0, z_0).$$

Splitting accumulators from both copies are divided by two. The physical Hamiltonian may vary with time; ΔK measures its computed balance. Tracking leaves physical trajectories, nonlinear stopping rules and adaptive acceptance unchanged. Dynamics must supply the momentum derivative, including zero for an autonomous Hamiltonian.

Hairer’s constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA’s simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired; `state_extension='fully_extended'` raises migration guidance. Dynamics, formulation, method and integration remain separate. The shared state classes own layouts, diagonal embedding and history extraction in

```
src/formulations/state.py
```

Each integration initializes a fresh method instance. Observers receive physical states; energy histories are in `Solution.diagnostics`. The `extended_time` history has shape (N, n_{samples}) . Fixed-step shadow samples leave main states and counters unchanged. Adaptive energy quadrature uses accepted dense output; its backend state and Jacobian remain physical.

7 Optional JAX execution

An optional JAX driver executes the same numerical map on one selected CPU or GPU, with particle operations batched inside a compiled sequential time loop. It reuses the canonical coefficients and prepared potential splines. Physical-state tolerance scaling, signed stage times, passive energy weights and independent shortened output maps retain their definitions above. Applicable Newton systems use independent particle blocks; an explicitly selected Broyden iteration retains its global secant matrix. Nonlinear iteration limits and failure checks remain active on the device.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.